EXERCICE 01 / 4

LE SERVEUR

Node.js natif · Modules · npm · HTTP

Difficulte : ★☆☆☆ 24 heures

Contexte pedagogique

Avant d'utiliser Express, l'apprenant doit comprendre ce que Node.js fait nativement. Dans cet exercice, on construit un mini-serveur HTTP SANS framework — avec uniquement les modules integres de Node.js. Cela permet de comprendre ce qu'Express simplifie, et pourquoi il existe. C'est le meme principe qu'en Section 1 : HTML pur avant CSS.

Competences ciblees

- Comprendre Node.js : moteur V8, event loop, nature non-bloquante
- Modules CommonJS : require, module.exports et les modules ES natifs
- Modules integres : http, fs, path, url, os
- npm : init, install, scripts, package.json, node_modules, .gitignore
- Creer un serveur HTTP natif : createServer, request, response
- Router manuellement les requetes : parsing de l'URL et de la methode
- Lire et ecrire des fichiers JSON comme base de donnees temporaire
- Variables d'environnement : fichier .env et module dotenv

CONFIGURATION MOODLE — Parametres de l'activite

```
Type d'activite   : Devoir (Assignment)
Titre             : EX01 — Le Serveur : Node.js natif & HTTP
Prerequis         : Section 2 validee — Badge React Developer obtenu
Delai             : 24 heures apres ouverture
Rendu             : Lien GitHub repo Node.js (branche main)
Note max          : 10 points
Note de passage   : 6/10 pour debloquer EX02
Ressources        : Video Node.js intro (20min) + Fiche modules natifs +
Doc officielle nodejs.org
```

Consigne detaillee

Creer un nouveau projet Node.js depuis zero. Construire un serveur HTTP qui repond a des requetes pour une mini-API de capteurs IoT — sans utiliser Express ni aucun framework. Toute la logique de routing est manuelle.

1 Initialisation du projet Node.js

Creer la structure du projet et configurer npm.

```
// Terminal — Initialisation projet Node.js
```

```
mkdir api-iot && cd api-iot
npm init -y
npm install dotenv

// Structure du projet :
api-iot/
  src/
    server.js        // Point d'entree
    router.js        // Routing manuel
    controllers/
      sensors.js     // Logique metier capteurs
    data/
      sensors.json   // Donnees temporaires (fichier JSON)
    utils/
      response.js    // Helpers pour les reponses HTTP
  .env               // Variables d'environnement
  .gitignore         // node_modules, .env
  package.json
```

- Le fichier .gitignore doit contenir node_modules/ et .env — jamais commiter ces fichiers
- package.json : ajouter le script start avec node src/server.js et dev avec nodemon

```
// JSON — package.json scripts
"scripts": {
  "start": "node src/server.js",
  "dev":   "nodemon src/server.js"
}
```

2 Serveur HTTP natif avec le module http

Creer le serveur HTTP de base sans aucun framework. Comprendre la requete et la reponse brutes.

```
// JavaScript — Serveur HTTP natif Node.js
// src/server.js
const http    = require('http');
const url     = require('url');
require('dotenv').config();

const PORT = process.env.PORT || 3000;

const server = http.createServer((req, res) => {
  // Parser l'URL pour extraire le chemin et les query params
  const parsedUrl = url.parse(req.url, true);
  const pathname  = parsedUrl.pathname;
  const method    = req.method;

  // Headers CORS et JSON pour toutes les reponses
  res.setHeader('Content-Type', 'application/json');
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE');

  // Routing manuel
  if (pathname === '/api/sensors' && method === 'GET') {
```

```
    handleGetSensors(req, res);
  } else if (pathname === '/api/sensors' && method === 'POST') {
    handlePostSensor(req, res);
  } else if (pathname.startsWith('/api/sensors/') && method === 'GET') {
    const id = parseInt(pathname.split('/')[3]);
    handleGetSensorById(req, res, id);
  } else {
    res.writeHead(404);
    res.end(JSON.stringify({ error: 'Route introuvable' }));
  }
});

server.listen(PORT, () => {
  console.log(`Serveur demarre sur http://localhost:${PORT}`);
});
```

- CORS (Cross-Origin Resource Sharing) : ces headers permettent au React Dashboard de faire des requetes vers l'API
- Observer que le routing manuel est verbeux — c'est pourquoi Express a ete cree

3 Lire le body d'une requete POST

En HTTP natif, le body d'une requete POST arrive par morceaux (chunks). Il faut les assembler.

```
// JavaScript — Lire le body d'une requete POST
// src/utils/parseBody.js
const parseBody = (req) => {
  return new Promise((resolve, reject) => {
    let body = '';
    req.on('data', chunk => { body += chunk.toString(); });
    req.on('end',  ()    => {
      try {
        resolve(JSON.parse(body));
      } catch (e) {
        reject(new Error('JSON invalide dans le body'));
      }
    });
    req.on('error', reject);
  });
};

module.exports = parseBody;

// Utilisation dans le handler POST :
const handlePostSensor = async (req, res) => {
  try {
    const body = await parseBody(req);
    // body = { name: 'Capteur 1', type: 'temperature', value: 25.3 }
    // ... traitement ...
  } catch (err) {
    res.writeHead(400);
    res.end(JSON.stringify({ error: err.message }));
  }
};
```

- req.on('data') : Node.js envoie le body par morceaux pour les grandes requetes
- Cette complexite disparait completement avec Express : req.body est deja parse

4 CRUD complet sur un fichier JSON

Implementer les 4 operations CRUD (Create, Read, Update, Delete) en lisant et ecrivant dans un fichier sensors.json.

```
// JavaScript — CRUD avec le module fs
// src/controllers/sensors.js
const fs   = require('fs');
const path = require('path');

const DATA_PATH = path.join(__dirname, '../data/sensors.json');

// Lire tous les capteurs
const getAllSensors = () => {
  const raw  = fs.readFileSync(DATA_PATH, 'utf-8');
  return JSON.parse(raw);
};

// Ajouter un capteur
const addSensor = (sensor) => {
  const sensors  = getAllSensors();
  const newSensor = { id: Date.now(), ...sensor, createdAt: new Date() };
  sensors.push(newSensor);
  fs.writeFileSync(DATA_PATH, JSON.stringify(sensors, null, 2));
  return newSensor;
};

// Supprimer un capteur par id
const deleteSensor = (id) => {
  const sensors  = getAllSensors();
  const filtered = sensors.filter(s => s.id !== id);
  if (filtered.length === sensors.length) return false; // Non trouve
  fs.writeFileSync(DATA_PATH, JSON.stringify(filtered, null, 2));
  return true;
};

module.exports = { getAllSensors, addSensor, deleteSensor };
```

- Date.now() genere un id unique base sur le timestamp en millisecondes
- Cette approche fichier JSON sera remplacee par PostgreSQL en EX04
- Tester l'API avec Postman ou curl avant de passer a EX02